Feature Extraction and Dimensionality Reduction

Week# 13

Course Instructor : Mahrukh Khan

Singular Value Decomposition (SVD)

- SVD also gives back principal components, but uses linear algebra and vector calculus to decompose the data matrix into three matrices which results in what are called singular values. The sum of the squares of the singular values should approximately equal the total variance in the original data matrix. We won't get into the technical aspects of it here as it isn't necessary to understand it in order to use it.
- Let *A* be any *m x n* matrix. Then the SVD divides this matrix into 2 unitary matrices that are orthogonal in nature and a rectangular diagonal matrix containing singular values till *r*. Mathematically, it is expressed as:
  - $A = U\Sigma V^T$ where,
  - **Σ ->** (m x n) diagonal matrix where the elements of the diagonal are the singular values
  - **U ->** (m x m) orthogonal matrix whose columns are the left singular vectors
  - **V ->** (n x n) orthogonal matrix whose columns are the right singular vectors

Application-SVD

- SVD is used for image compression and denoising. It helps in reducing the dimensionality of image data by preserving the most significant singular values and discarding the rest.
- Original image = combination of patterns × strength of each pattern × orientation

Take a small 2×2 matrix:

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

Compute:

1. $A^T A$
2. Eigenvalues → get Σ
3. Eigenvectors → get V
4. Compute $U = AV/\Sigma$
5. Verify $A = U\Sigma V^T$

- Step 1:

Compute $A^T A$

$$A^T A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}^T \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

$$A^T A = \begin{bmatrix} 3(3)+1(1) & 3(1)+1(3) \\ 1(3)+3(1) & 1(1)+3(3) \end{bmatrix} = \begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix}$$

- Step 2:

Find eigenvalues of $A^T A$

$$\det(A^T A - \lambda I) = 0$$

$$\det \begin{bmatrix} 10-\lambda & 6 \\ 6 & 10-\lambda \end{bmatrix} = (10-\lambda)^2 - 36 = 0$$

$$(10-\lambda)^2 = 36 \Rightarrow 10-\lambda = \pm 6$$

$$\lambda_1 = 16, \quad \lambda_2 = 4$$

- Step 3:

Compute singular values

$$\sigma_i = \sqrt{\lambda_i}$$

$$\sigma_1 = 4, \quad \sigma_2 = 2$$

$$\Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}$$

- Step 4:Find eigenvectors of $A^T A$)for V)

For $\lambda_1 = 16$:

$$(A^T A - 16I)v = 0 \Rightarrow \begin{bmatrix} -6 & 6 \\ 6 & -6 \end{bmatrix} v = 0$$

For $\lambda_2 = 4$:

$$(A^T A - 4I)v = 0 \Rightarrow \begin{bmatrix} 6 & 6 \\ 6 & 6 \end{bmatrix} v = 0$$

- Step 5:Find Uusing $U = \frac{AV}{\Sigma}$

$$Av_1 = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 4 \\ 4 \end{bmatrix} \Rightarrow u_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$Av_2 = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 2 \\ -2 \end{bmatrix} \Rightarrow u_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}$$

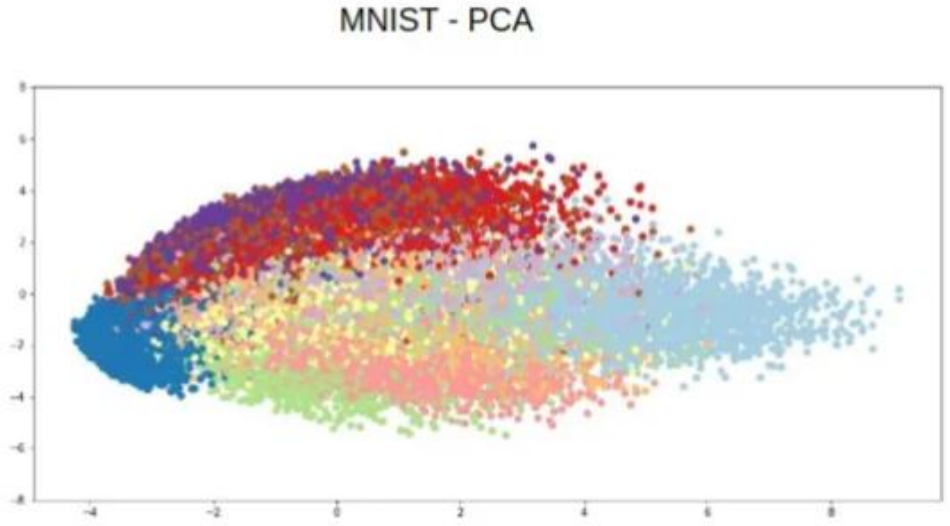
- Step 6: Verify $A = U\Sigma V^T$

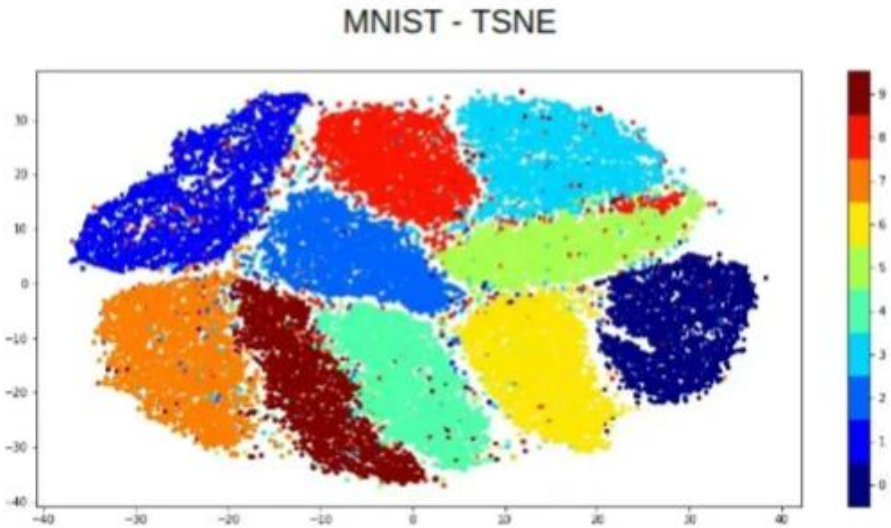
$$U\Sigma V^T = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}^T$$

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

t-SNE

- t-Distributed Stochastic Neighbour Embedding is a statistical method for visualizing high-dimensional data by reducing it to lower -dimensional spaces, typically two or three dimensions.
- makes it easier to visualize and interpret the data, especially when dealing with complex datasets.





How t-SNE works

- It achieves this by measuring the similarity between data points in the high-dimensional space and representing this similarity as probabilities. Then, it constructs a similar probability distribution in the lower-dimensional space and minimizes the difference between the two distributions using a technique called gradient descent.
- This way, t-SNE focuses on preserving the local structure and pattern of the data.
- https://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf

- **In high dimension->**It converts distances into **probabilities**:
  - $p_{j|i}$ =probability that point *j* is a neighbor of *i*.
  - Uses a Gaussian distribution to define this.
- **In low dimension (2-D)->**It defines similar probabilities $q_{j|i}$using a **Student-t distribution** (fatter tails)
- Minimize the difference between P(high-D similarities) and Q(low-D similarities) using **Kullback-Leibler divergence (KL)**.
- That's how it "pulls" similar points together and "pushes" dissimilar ones apart.

```
class sklearn.manifold.TSNE(n_components=2, *, perplexity=30.0, early_exaggeration=12.0,
learning_rate='auto', max_iter=1000, n_iter_without_progress=300, min_grad_norm=1e-07,
metric='euclidean', metric_params=None, init='pca', verbose=0, random_state=None,
```

- **n_components***int, default=2*
- Dimension of the embedded space.
- **perplexity***float, default=30.0*
- The perplexity is related to the number of nearest neighbors that is used in other manifold learning algorithms. Larger datasets usually require a larger perplexity. Consider selecting a value between 5 and 50. Different values can result in significantly different results. The perplexity must be less than the number of samples.
- Perplexity is a measure related to the **effective number of neighbors** considered for each point when t-SNE computes pairwise similarities in high-dimensional space.
- **max_iter***int, default=1000*
- Maximum number of iterations for the optimization. Should be at least 250.

Applications Of t-SNE

- **Clustering and classification:** to cluster similar data points together in lower dimensional space. It can also be used for classification and finding patterns in the data.
- **Anomaly detection:** to identify outliers and anomalies in the data.
- **Natural language processing:** to visualize word embeddings generated from a large corpus of text that makes it easier to identify similarities and relationships between words.
- **Computer security:** to visualize network traffic patterns and detect anomalies.
- **Cancer research:** to visualize molecular profiles of tumor samples and identify cancer subtypes.
- **Geological domain interpretation:** to visualize seismic attributes and to identify geological anomalies.
- **Biomedical signal processing:** to visualize electroencephalogram (EEG) and detect patterns of brain activity.

Limitations and Challenges of t-SNE

- **Computational cost**: t-SNE is computationally expensive, especially for large datasets. Its pairwise similarity calculations scale poorly with the size of the dataset, making it less suitable for datasets with millions of points.
- **Sensitivity to hyperparameters**: The performance of t-SNE is highly dependent on hyperparameters like perplexity and learning rate. Finding the optimal values often requires trial and error, which can be time-consuming.
- **Lack of interpretability**: The resulting embeddings in t-SNE are often non-deterministic (due to random initialization and gradient descent) and lack a direct interpretable relationship with the original high-dimensional data.
- **Not suitable for out-of-sample data**: t-SNE cannot generalize to new, unseen data without re-computing the embeddings from scratch, which is inefficient for dynamic datasets.

Technique	Type	Supervised	Preserves	Best for
PCA	Linear	✗	Global	Compression
LDA	Linear	✓	Class separation	Classification
SVD	Linear Algebra	✗	Matrix structure	Recommenders
t-SNE	Non-linear	✗	Local structure	Visualization

Confusion Metrics

Actual/Predicted	Pred. A	Pred. B	Pred. C	Support(Actual)
A	30	5	5	40
B	3	25	2	30
C	2	4	24	30
Predicted Totals	35	34	31	100

Step 1: Per class Counts

- Class A
- TP_A=30(actual A Predicted A)
- FP_A=Predicted_A_Total-TP_A=35-30=5
- FN_A=Actual_A_Total-TP_A=40-30=10
- Class A
- TP_B=25(actual B Predicted B)
- FP_B=Predicted_B_Total-TP_B=34-25=9
- FN_**B**=Actual_B_Total-TP_B=30-25=5
- Class C
- TP_C=24(actual C Predicted C)
- FP_C=Predicted_C_Total-TP_C=31-24=7
- FN_C=Actual_C_Total-TP_C=30-24=6

Precision_A=TP/TP+FP

For Class 30/30+5=0.857

Recall_A

Example-2

Actual \ Predicted	A	B	C	Total Actual
A	40	5	5	50
B	3	45	2	50
C	4	6	40	50
Total Predicted	47	56	47	150

Accuracy

- **Accuracy**
- $\text{Accuracy} = \frac{\textit{Correct Predictions}}{\textit{Total Samples}} = \frac{40+45+40}{150} = \frac{125}{150} = 0.833$
- ✓ **Accuracy = 83.3%**

Precision

- Precision for a class = Correctly predicted of that class ÷ All predicted as that class

Class	True Positives (TP)	Predicted Total	Precision
A	40	47	40/47 = **0.851**
B	45	56	45/56 = **0.804**
C	40	47	40/47 = **0.851**

Recall

- Recall for a class = Correctly predicted of that class ÷ All actual of that class

Class	True Positives (TP)	Actual Total	Recall
A	40	50	40/50 = **0.80**
B	45	50	45/50 = **0.90**
C	40	50	40/50 = **0.80**

F1 Score

- $F1=2\times(Precision \times Recall)(Precision+Recall)$

Class	Precision	Recall	F1-score
A	0.851	0.80	**0.825**
B	0.804	0.90	**0.849**
C	0.851	0.80	**0.825**

Average Type	Formula Idea	Equal Weight?	Sensitive to Class Imbalance?	Use Case
Macro	Mean of per-class metrics	✓ Yes	✗ No (treats all equally)	To see performance per class (fair comparison)
Micro	Global count-based metric	✗ No	✓ Yes (dominated by large classes)	Overall model performance
Weighted	Weighted by #samples per class	✗ No	✓ Yes	Balanced performance measure

Macro Average

- Average of the metric across all classes **equally**, regardless of how many samples each class has.
- Macro = average of metrics per class)
- Macro $Precision = \frac{(P_A + P_B + P_C)}{3}$
- ✓ Every class has **equal weight**.
  ✓ Reflects how the model performs **per class**, even for small ones.
  ✗ Can misrepresent overall performance if classes are highly imbalanced.
  - Macro $Precision = (0.851 + 0.804 + 0.851)/3 = 0.835$
  - Macro $Recall = (0.80 + 0.90 + 0.80)/3 = 0.833$
  - Macro $F1 = (0.825 + 0.849 + 0.825)/3 = 0.833$

Micro Average

- Aggregate the contributions of **all classes** to compute a single metric. Counts all **true positives, false negatives, and false positives** globally.
- Micro = total true positives / total predictions)
- Micro $Precision = Micro\ Recall = \frac{\sum TP}{Total\ Samples} = \frac{125}{150} = 0.833$
- Since micro averages are same:
- Micro $F1 = 0.833$
- ✓ **Micro Precision = Recall = F1 = 0.833**
- Same as **overall accuracy** when classes are disjoint.
  ✓ Good for overall system performance.
  ✗ Dominated by majority class if data is imbalanced.

Weighted Averages

- Weighted average of per-class metrics, **weighted by the number of true instances (support)** of each class.
- (Weighted = average weighted by class size)
- All classes have equal samples (50 each), so:
- Weighted $Avg = Macro\ Avg = 0.833$
- ✓ Reflects the **true data distribution**.
  ✓ Good when class sizes are very different.
  ✗ Can hide poor performance on rare classes if they're few.

Scenario	Best Average
Balanced classes	All three give similar results
Imbalanced classes (some rare)	Use weighted average
Want to check fairness between classes	Use macro average
Want overall global accuracy-like measure	Use micro average